# Получение данных из Excel и API

Материал продолжает работу со знакомыми ипотечными записями HMDA. Новые понятия разбираются на небольших таблицах и сохранённых ответах. Демонстрационный ноутбук содержит полный код; здесь собраны объяснения и короткие примеры.

## Слайд 1. Получение данных

На прошлом занятии мы добавляли названия из готового справочника. Теперь разберёмся, как такой справочник получить. Сначала откроем Excel-книгу, затем прочитаем JSON — текстовый формат, в котором сервисы часто возвращают данные. После этого составим запрос к API и проверим ответ.

API — application programming interface, интерфейс для обращения одной программы к другой. Мы отправляем запрос по определённому адресу и получаем ответ с данными. В качестве примера используем GLEIF, реестр юридических лиц с кодами LEI. Выбраны три кода, которые уже встречаются в нашем HMDA. Ответы были сохранены 10 октября 2026 года, поэтому занятие можно выполнить без интернета. Они показывают сведения реестра на момент получения, а ипотечные записи относятся к 2023 году.

## Слайд 2. Формат и способ получения

Один набор сведений можно получить разными способами и сохранить в разных форматах. CSV хранит строки и столбцы в тексте. Excel-книга может содержать несколько листов. JSON записывает вложенные словари и списки. API определяет, как программа запрашивает сведения: какой адрес вызвать, какие параметры передать и как читать ответ.

Например, справочник HMDA Filers получен в JSON. Для упражнения мы также сохранили его в Excel на лист organizations. Это те же сведения в другом представлении. GLEIF возвращает JSON со списком data. На практике сохраним нужные поля в CSV. Иногда для больших таблиц используют Parquet: он хранит данные по столбцам и сохраняет их типы. Сегодня с Parquet работать не требуется; сначала освоим чтение знакомых таблиц и JSON.

| Представление | Что внутри |
|---|---|
| CSV | Одна плоская таблица |
| Excel | Книга с листами |
| JSON | Вложенные словари и списки |
| API | Правила обращения программы к сервису |

## Слайд 3. Excel-книга и выбор листа

В книге hmda_exchange.xlsx два листа. organizations содержит справочник организаций, reading_example — десять ипотечных записей. Прежде чем выбирать лист, выясняем, что означает его строка. Эти таблицы относятся к разным сущностям и имеют разное количество строк.

pd.read_excel читает книгу. При sheet_name=None он возвращает словарь: ключи — названия листов, значения — DataFrame. Мы напечатаем названия и размеры. Затем

укажем sheet_name="organizations" и получим одну таблицу. usecols оставляет lei, name и period; dtype={"lei": "string"} сохраняет код текстом. period здесь обозначает отчётный год, name — опубликованное имя. Найдём эти поля в первых строках, проверим известность кода и уникальность пары lei/period.

```
book = pd.read_excel(
    LOCAL / "hmda_exchange.xlsx", sheet_name=None
)
print({name: table.shape for name, table in book.items()})
```

## Слайд 4. Как устроен JSON

Посмотрим на небольшой фрагмент записи. Фигурные скобки окружают объект JSON: внутри находятся пары «ключ — значение». После чтения в Python такой объект станет dict, словарём. Квадратные скобки задают массив; в Python это list. Список data может содержать несколько словарей организаций.

Строки JSON записываются в двойных кавычках. Число остаётся числом, null превращается в None, true — в True. Вложенность означает, что значением словаря может быть другой словарь или список. Например, у записи есть attributes, внутри него entity, затем legalName, а внутри legalName — name. Чтобы получить название, нужно пройти этот путь. Сначала прочитаем файл целиком, затем найдём одну запись и её имя.

На слайде оставлены код и имя первой реальной записи; остальные поля опущены для чтения. Откройте полный файл, когда перейдём к следующему шагу.

```
{"data": [
  {"id": "549300P1347BFCBHJV60",
   "attributes": {"entity": {
     "legalName": {"name": "MASS. INSTITUTE OF TECH. FEDERAL CREDIT UNION"}
   }}}
]}
```

## Слайд 5. Чтение сохранённого ответа

Откроем gleif_page_01.json с кодировкой UTF-8. Конструкция with закрывает файл после чтения. json.load(file) разбирает его содержимое и возвращает Python-объект. В нашем случае это словарь. Напечатаем его тип и ключи: data содержит записи, meta — сведения о выдаче, links — ссылки, в том числе продолжение результата.

Для второго примера прочитаем filers_2023_dc.json как текст через read_text и передадим текст в json.loads. Отличие одной буквы s полезно запомнить: loads получает строковое содержимое, load — открытый файл. Сам путь к файлу содержимым JSON не является. Также прочитаем manifest: это сопровождающий файл с адресами, временем получения и контрольными суммами сохранённых ответов. К нему вернёмся при проверке полноты.

```
with (LOCAL / "gleif_page_01.json").open(encoding="utf-8") as file:
    page1 = json.load(file)
print(type(page1), list(page1))
```

## Слайд 6. Одна запись внутри ответа

Пройдём по ответу по одному шагу. page1 — словарь. page1["data"] возвращает список записей, [0] выбирает первую запись. В record["id"] находится её LEI. Дальше record["attributes"] — словарь атрибутов; entity описывает юридическое лицо; legalName содержит имя; name возвращает строку названия.

Квадратные скобки с ключом требуют его наличия. Если обязательного ключа нет, Python выдаст KeyError. Метод get позволяет получить необязательное значение:

при отсутствии ключа вернётся None, если другой вариант не задан. Поэтому следующий адрес удобно читать как page1["links"].get("next"). Обязательные data и id проверим явно. Если они исчезли, продолжать расчёт как с пустой таблицей нельзя: изменилось устройство ожидаемого ответа.

```
record = page1["data"][0]
record["id"]
record["attributes"]["entity"]["legalName"]["name"]
```

## Слайд 7. Список записей превращается в таблицу

Вернёмся к справочнику HMDA Filers. Его корень тоже словарь, но список организаций находится под ключом institutions. Метод pd.json_normalize умеет преобразовать такой список в таблицу. Через record_path="institutions" указываем, откуда брать записи для строк.

После вызова получим 470 строк и обычные столбцы lei, name, period, count. Найдём код, имя и год в первых строках. Проверим, что пара lei/period не повторяется. count — счётчик из отбора самого источника. Для долей по нашим ипотечным записям знаменатель берём из decisions, как на предыдущей лекции. Сейчас задача normalize состоит в выборе записей и переносе их полей в табличную форму; аналитический расчёт выполняется отдельно.

```
orgs = pd.json_normalize(filers, record_path="institutions")
orgs.head(3)
```

## Слайд 8. Вложенные поля становятся столбцами

В GLEIF сначала берём page1["data"], список словарей организаций, и передаём его в json_normalize. sep="." задаёт разделитель уровней. Поэтому путь attributes → entity → legalName → name станет одним длинным названием столбца: attributes.entity.legalName.name.

Рядом выберем id и attributes.entity.legalAddress.country. Первый столбец — LEI, второй — юридическое имя, третий — страна юридического адреса организации. Эта страна описывает адрес организации; территория ипотечной записи по-прежнему хранится в county_code исходного HMDA. Покажем две строки первой страницы и прочитаем их как обычную таблицу. Поле otherNames пока останется списком внутри клетки: для списка может потребоваться отдельное решение о том, что должна означать строка.

```
flat1 = pd.json_normalize(page1["data"], sep=".")
cols = ["id", "attributes.entity.legalName.name",
        "attributes.entity.legalAddress.country"]
flat1[cols]
```

## Слайд 9. Список других имён

Внутри entity есть otherNames, список других названий организации. Если развернуть этот список, строка таблицы будет описывать уже одно имя. У одной организации таких строк может оказаться несколько. Число строк поэтому может отличаться от числа юридических лиц.

Для такого разбора json_normalize получает record_path=["attributes", "entity", "otherNames"]. Этот путь ведёт к списку имён внутри каждой записи. meta=["id"] переносит LEI в каждую получившуюся строку, чтобы имя можно было связать с организацией. Покажем первые строки и посчитаем, сколько имён было в исходных списках. Такой результат можно хранить как отдельную таблицу имён. Для

справочника с одним именем на LEI будем использовать legalName, уже полученное на прошлом шаге.

```
aliases = pd.json_normalize(
    page1["data"],
    record_path=["attributes", "entity", "otherNames"],
    meta=["id"]
)
```

## Слайд 10. Проверка полученной таблицы

Перед использованием таблицы проверим, есть ли в ней нужные поля: id, юридическое имя и страна адреса. required — множество ожидаемых названий. issubset проверяет, что все они входят в набор столбцов; дополнительные поля при этом допустимы.

Затем проверим notna().all() для id и свойство is_unique. Так узнаем, что в каждой строке есть код и он не повторяется. Но пока мы читали только первую страницу: в ней две организации, а meta.pagination.total сообщает три записи для нашего запроса. Схема первой страницы корректна, однако полный набор ещё не получен. Поэтому в итоговом отчёте разделим проверки структуры и проверки количества запрошенных организаций.

```
assert required.issubset(flat1.columns)
assert flat1["id"].notna().all()
assert flat1["id"].is_unique
```

## Слайд 11. Запрос GET и его параметры

Чтобы получить сведения от сервиса, программе нужен адрес. Здесь endpoint указывает на список записей LEI в GLEIF. Запрос GET означает чтение ресурса. В params передадим словарь параметров: filter[lei] ограничит ответ тремя выбранными кодами, page[size] задаст желаемое число записей на странице.

Коды берём из manifest["selected_lei"], а join через запятую превращает список строк в одну строку параметра. requests.Request с prepare соберёт адрес, но не отправит запрос. Это удобный способ рассмотреть его без сети. Библиотека закодирует квадратные скобки и запятые в URL. Напечатаем адрес и сравним его с сохранённым первым запросом. Для HTTP-запросов используется библиотека requests; она уже включена в окружение курса.

```
params = {"filter[lei]": ",".join(selected), "page[size]": 2}
prepared = requests.Request("GET", endpoint, params=params).prepare()
print(prepared.url)
```

## Слайд 12. Выполнение запроса requests.get

Сетевой запрос выполняет requests.get. Первый аргумент — адрес, params содержит условия, headers задаёт заголовки запроса. Через Accept сообщаем, какой формат ожидаем получить. timeout=(5, 30) задаёт два ограничения: ожидание установления соединения и ожидание данных при чтении. Это не общий предел длительности всех возможных загрузок.

В учебном ноутбуке вызов находится внутри if RUN_LIVE, а переключатель установлен в False. Поэтому по умолчанию сеть не используется и мы продолжаем разбор сохранённого ответа. Сетевой пример можно выполнить отдельно. Он может вернуть другие сведения, поскольку реестр обновляется. Для воспроизведения нашего занятия оставляем сохранённые файлы и даты их получения без изменений.

```
RUN_LIVE = False
if RUN_LIVE:
    live_response = requests.get(
        endpoint, params=params, timeout=(5, 30)
    )
```

## Слайд 13. Что содержит HTTP-ответ

После запроса requests возвращает Response. status_code — числовой статус HTTP: например, 200 означает успешный ответ на этом уровне. headers — заголовки ответа; Content-Type сообщает формат тела. content содержит исходные байты. Метод json() пытается разобрать тело как JSON и возвращает словарь или список Python.

Сегодня для демонстрации соберём объект Response из сохранённых метаданных и файла. Это воспроизведение прошлого ответа, а не новый вызов сервиса. Поле _content здесь задаётся вручную только для такого учебного восстановления. raise_for_status() проверяет HTTP-статус и вызывает исключение для ошибочных ответов 4xx и 5xx. После него сравним результат json() с уже прочитанным page1. Успешный статус и разбор JSON проверяют разные свойства ответа.

```
print(response.status_code)
print(response.headers.get("Content-Type"))
response.raise_for_status()
payload = response.json()
```

## Слайд 14. Ошибка сервиса и следующий шаг

Создадим отдельную копию восстановленного ответа и поставим ей статус 429. Он сообщает об ограничении частоты запросов. raise_for_status остановит такую обработку исключением HTTPError. Пустой список организаций вместо ошибки скрыл бы разницу между «ничего не найдено» и «данные не удалось получить».

Для 400 нужно проверить параметры запроса; для 404 — адрес и существование ресурса. При 429 учитывают ограничение сервиса и заголовок Retry-After, если он есть. При временных сбоях сервера или соединения возможен ограниченный повтор. На занятии мы обнаруживаем ошибку и сообщаем о ней. Полноценные правила повторов относятся к отдельной задаче: бесконечное повторение одного вызова не помогает понять, получен ли результат.

| Статус или сбой | Что проверить |
|---|---|
| 200 | Формат, структуру и полноту тела |
| 400 / 404 | Параметры и адрес |
| 429 | Ограничение частоты, Retry-After |
| 5xx / ошибка соединения | Причину сбоя и допустимость повтора |

## Слайд 15. Проверки ответа по порядку

Объединим первые проверки в decode_json_response. Сначала raise_for_status проверяет HTTP-статус. Потом читаем Content-Type. Для нашего источника ожидается application/json или application/vnd.api+json. Заголовок может содержать дополнительные параметры после точки с запятой; split отделяет сам тип.

Затем json() разбирает тело. Проверим, что корень — словарь и внутри есть список data. isinstance позволяет проверить тип Python-объекта. Если формат или структура не подходят, функция выдаёт ValueError с причиной. Вызовем её на сохранённом

ответе, затем на учебной копии, в которой Content-Type заменён на text/html. Код 200 у копии останется, но проверка формата её остановит. Это конкретный случай, когда одного HTTP-статуса недостаточно.

```
payload = decode_json_response(response)
print(type(payload), list(payload))
```

## Слайд 16. Результат на нескольких страницах

Сервис не обязан возвращать все найденные записи в одном ответе. Он может разделить их на страницы; такой способ называют пагинацией. В нашем запросе размер страницы равен двум, а выбранных LEI три. Поэтому первую пару получили на первой странице, последнюю организацию — на второй.

Откроем page2 и сравним meta.pagination у обеих страниц. Затем прочитаем links.next первой: ссылка должна совпасть с адресом сохранённого второго запроса. У второй страницы next отсутствует или равен None — продолжения нет. Сложим два списка data и проверим набор полученных id. Ожидаем ровно те три кода, которые запрашивали. Число три само по себе не доказывает, что пришли нужные организации, поэтому сравним также множества кодов.

```
page1["links"].get("next")
page2["links"].get("next")
all_records = page1["data"] + page2["data"]
```

## Слайд 17. Обход страниц в одной функции

На двух страницах переходы легко выполнить вручную. Если их больше, удобно сохранить порядок в collect_pages. В неё передаём начальный адрес и loader — функцию, которая получает страницу по адресу. Наш cached_loader берёт ответ из локального словаря cache, поэтому обход работает без сети.

Внутри collect_pages url хранит следующий адрес, records накапливает записи, seen_urls запоминает уже открытые ссылки. На каждом шаге функция читает страницу, проверяет список data и ожидаемое total, добавляет записи через extend и берёт следующий адрес из links. Если адрес повторился, возникает ошибка цикла. max_pages ограничивает число шагов; достижение предела при оставшемся next означает незавершённую загрузку. В конце сравниваются количество записей и уникальность id. Прочитаем готовую функцию этими блоками, затем выполним один вызов.

```
collected = collect_pages(manifest["first_url"], cached_loader)
assert {r["id"] for r in collected} == set(selected)
```

## Слайд 18. Сохранённые ответы и manifest

После загрузки сохраняем тело каждого ответа как полученные байты, до выбора полей и переименования. Такие исходные файлы часто называют raw. Рядом находится manifest: в нём записаны адрес запроса, время получения, HTTP-статус, Content-Type и контрольная сумма файла.

SHA-256 вычисляет по байтам строку контрольной суммы. Если файл изменится, совпадение с сохранённой суммой практически наверняка нарушится. Прочитаем каждый файл через read_bytes, вычислим hashlib.sha256(body).hexdigest() и сравним со значением в manifest. Так проверяем, что разбираем именно сохранённый ответ. Истинность сведений и полноту набора мы проверяем другими способами: по происхождению, структуре, числу записей и кодам. Новый ответ сохраняем отдельно от учебного снимка.

```
body = (LOCAL / info["file"]).read_bytes()
assert hashlib.sha256(body).hexdigest() == info["sha256"]
```

## Слайд 19. Добавление сведений GLEIF

Преобразуем collected в плоскую таблицу и выберем LEI, юридическое имя и статус лица. Переименуем поля в lei, legal_name_current и entity_status_current. Суффикс current напоминает: это сведения на момент сохранения реестра, а не готовые исторические значения за 2023 год.

Затем выполним знакомое left-соединение с decisions по lei. Справа один код встречается один раз, поэтому проверяем many_to_one. Все 6830 ипотечных записей остаются. У 76 найдутся сведения трёх выбранных организаций, у 6754 будет left_only. Такой частичный охват ожидаем: запрос был ограничен тремя кодами. Неизвестный добавленный статус у остальных записей означает, что они не вошли в этот запрос. Исходные семь полей сравним с decisions, чтобы проверить их сохранность.

```
enriched = decisions.merge(
    lookup, on="lei", how="left",
    validate="many_to_one", indicator=True
)
enriched["_merge"].value_counts()
```

## Слайд 20. Год записи и время получения справочника

В activity_year указано, к какому отчётному году относится ипотечная запись. downloaded_at_utc сообщает, когда мы сохранили ответ API. golden_copy_publish_date — время публикации версии справочника. Эти даты описывают разные события.

Переведём две временные колонки manifest в даты через pd.to_datetime. utc=True приводит их к UTC, errors="raise" останавливает чтение при неразбираемой дате. Проверим, что публикация версии не позже её получения и что обе страницы относятся к одной опубликованной версии.

Если позже будем прогнозировать решение по заявке в 2023 году, понадобится информация, доступная к моменту прогноза. Статус из ответа 2026 года мог измениться после решения. Сегодня используем эти сведения как справочную иллюстрацию и показываем дату рядом с результатом.

```
pd.to_datetime(acquisition["downloaded_at_utc"], utc=True)
pd.to_datetime(acquisition["golden_copy_publish_date"], utc=True)
```

## Слайд 21. Повторный запуск и новая загрузка

Повторить анализ и заново обратиться к сервису — разные действия. При повторении мы читаем те же сохранённые файлы и проверяем их контрольные суммы. Такой запуск помогает сравнивать код и результаты. Новое обращение к API может дать другой ответ, потому что данные сервиса обновляются.

Поэтому для нового получения создаём отдельную папку и новый manifest. Сравниваем кодовый состав, количество записей, структуру и даты. Если сеть недоступна или запрос завершился ошибкой, сохраняем причину и не выдаём её за успешный пустой результат. Для нашей практики достаточно уже полученного снимка. В условии сдачи будет указано, какие исходные файлы, преобразованная таблица и сведения о получении должны сопровождать код.

| Действие | Какие файлы используем |
|---|---|
| Повтор анализа | Те же ответы и manifest |
| Новое получение | Новая папка ответов и новый manifest |
| Сбой получения | Сообщение с причиной; прежние файлы сохранены |

## Слайд 22. Вопрос учебного проекта

На практике начнём формулировать вопрос командного проекта. Возьмём возможный описательный вопрос: как различаются доли отказов между организациями в нашем срезе? Для него нужны исходы, код организации и количество записей. Модель машинного обучения для такого первого сравнения не требуется.

В предложении проекта укажем, для кого готовим ответ, что означает строка, какие записи включаем и как считаем показатель. Для доли отказов числитель — количество записей с кодом 3, знаменатель — количество всех выбранных решений организации. Затем назовём ограничение: организации могут различаться по составу записей и размеру групп. Первым шагом проверим эти размеры и распределения известных полей. Это помогает выбрать выполнимый анализ. В готовом шаблоне вы сформулируете свой вопрос и уточните его после обсуждения с другой командой.

```
proposal["question"]
proposal["population"]
proposal["metric"]
```

## Слайд 23. Практика получения данных

Повторим путь на одном примере. Из файла или ответа мы сначала получаем содержимое. Затем разбираем JSON, выбираем список для строк и нужные поля. Для сетевого ответа отдельно проверяем статус и формат. Если результат разбит на страницы, получаем их все и сверяем запрошенные коды. После этого сохраняем исходные ответы, таблицу и сведения о получении.

На практике P05 эти шаги представлены готовыми примерами и заданиями A–E. В конце подготовите короткое предложение проекта. Условия находятся в методичке, объяснения записывайте в отчёт рядом с ноутбуком. Домашний ноутбук отдельно загружает исходники и готовые вспомогательные функции. Сеть для обязательной части не нужна; дополнительные проверки испорченных копий страниц выполняются по желанию.

## Источники и повторение

Словарь и происхождение данных (../../../practices/P05/data/card.md). Общий снимок HMDA (../../../data/card.md).

Python json (https://docs.python.org/3.12/library/json.html), read_excel (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.read_excel.html), json_normalize (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.json_normalize.html), Requests (https://requests.readthedocs.io/en/latest/user/quickstart/), GLEIF API (https://api.gleif.org/docs).